UNIVERSIDAD AMERICANA

Facultad de Ciencias y Tecnología

Ingeniería en Sistemas / Ciencias de la Computación

TAREA PRÁCTICA · UNIDAD III

Conversión por rastreo

Implementación y extensión de los algoritmos DDA y Bresenham

Entrega vía repositorio Git + enlace en Canvas

Asignatura

Taller de Programación de 5.ª Generación I

Docente: Prof. Mgtr. Alberto F. Giménez Méndez

Asunción, Paraguay · 2026

Presentación de la tarea

Esta tarea práctica está diseñada para consolidar los conceptos trabajados en la clase sobre **conversión por rastreo**, particularmente los algoritmos **DDA** y **Bresenham** para el trazado de líneas sobre el frame buffer. El estudiante deberá reutilizar los códigos fuente demostrados en clase, extenderlos con creatividad y entregar el trabajo a través de un repositorio de control de versiones.

> 📋 **Datos generales de la tarea**
>
> Modalidad: individual.
>
> Duración estimada: 4 a 6 horas de trabajo.
>
> Puntaje: 20 puntos (dos ejercicios de 10 puntos cada uno).
>
> Entrega: enlace al repositorio Git + imágenes generadas en documento Word, comprimido y subido a Canvas.

Objetivos de aprendizaje

Al finalizar esta tarea el estudiante estará en condiciones de:

- Aplicar los algoritmos DDA y Bresenham estudiados en clase sobre un frame buffer manual implementado con Pillow.
- Diseñar composiciones gráficas más complejas que las vistas en clase a partir de las primitivas elementales.
- Documentar su código fuente y publicarlo en un sistema de control de versiones (Git).
- Entregar un trabajo profesional según los estándares de la industria del software.

Materiales de referencia

Para resolver esta tarea el estudiante dispone de los siguientes recursos trabajados en clase:

- Material de lectura de la Unidad III (Conversión por rastreo), secciones 1 y 2.
- Códigos fuente de las funciones `linea_directa()`, `dda()` y `bresenham()` presentados durante la Clase 6.
- Scripts de demostración ejecutados en clase: asterisco de 8 puntas, estrella de 5 puntas, benchmark DDA vs Bresenham.

Ejercicio 1 — Composición geométrica con DDA

Puntaje: 10 puntos **Algoritmo requerido:** DDA

Enunciado

Utilizando la función `dda()` presentada en clase, construya una composición geométrica que represente **una casa estilizada** (vista frontal) sobre un lienzo de 600 × 500 píxeles. La composición debe incluir, como mínimo, los siguientes elementos:

- Un **rectángulo** que represente el cuerpo de la casa (4 líneas).
- Un **triángulo** que represente el techo (3 líneas).
- Una **puerta rectangular** (4 líneas).
- Al menos **dos ventanas** cuadradas (8 líneas en total).
- Un **sol** en una esquina superior, representado con al menos 8 líneas que irradien desde un punto central.
- Una **línea de piso** que atraviese toda la imagen horizontalmente.

Requisitos técnicos

1. Debe reutilizar la función `dda()` tal como fue presentada en clase, sin modificar su firma.
2. Cada elemento gráfico (rectángulo, triángulo, puerta, ventanas, sol) debe estar implementado como una función Python separada, con nombre descriptivo (por ejemplo: dibujar_rectangulo, dibujar_triangulo, dibujar_sol).
3. Cada elemento debe tener un color distinto. Use al menos 4 colores diferentes en la composición.
4. El archivo final debe llamarse `ejercicio1_casa.py` y debe generar como salida la imagen `casa.png`.
5. El código debe estar comentado explicando cada bloque.

Pista de arranque

Estructura recomendada del archivo:

```
# ejercicio1_casa.py
from PIL import Image

def dda(pixels, x0, y0, x1, y1, color, ancho, alto):
    # ... (copiar la implementación vista en clase)

def dibujar_rectangulo(pixels, x0, y0, x1, y1, color, ancho, alto):
    # cuatro llamadas a dda() para los cuatro lados
    ...
```

```
def dibujar_triangulo(pixels, p1, p2, p3, color, ancho, alto):
    # tres llamadas a dda() conectando los tres vértices
    ...

def dibujar_sol(pixels, cx, cy, radio, n_rayos, color, ancho, alto):
    # bucle con cálculo polar y llamadas a dda()
    ...

# Programa principal
ancho, alto = 600, 500
imagen = Image.new("RGB", (ancho, alto), (200, 230, 255))
pixels = imagen.load()

# Llamar a cada función para construir la casa
dibujar_rectangulo(pixels, ...)
# ... etc

imagen.save("casa.png")
```

Resultado esperado

Una imagen PNG de 600 × 500 píxeles que represente una casa reconocible, con los seis elementos solicitados. La casa no tiene que ser realista — puede ser estilo infantil o esquemático —, pero debe demostrar que domina el uso de la función dda() para construir composiciones.

Extensiones voluntarias (hasta 2 puntos adicionales)

Agregue árboles al paisaje (tronco rectangular + copa triangular).

Agregue nubes compuestas por múltiples líneas cortas.

Agregue humo saliendo de una chimenea.

Experimente con gradientes de color en elementos distintos.

Ejercicio 2 — Composición dinámica con Bresenham

Puntaje: 10 puntos **Algoritmo requerido:** Bresenham

Enunciado

Utilizando la función `bresenham()` presentada en clase, implemente un **generador de rosetas geométricas** (patrones radiales inspirados en espirógrafos o mandalas). La roseta debe generarse mediante la conexión programática de puntos distribuidos sobre una circunferencia imaginaria.

Especificaciones de la roseta

- Lienzo de **700 × 700 píxeles** con fondo blanco o negro (a elección del alumno).
- Distribuya **N puntos equiespaciados** sobre una circunferencia imaginaria centrada en el lienzo (sin dibujar la circunferencia). Use N = 24 como valor base.
- Conecte **cada punto con todos los demás** mediante líneas trazadas con `bresenham()`. Para N = 24, se dibujarán 276 líneas en total.
- Aplique un **gradiente de color** a las líneas: el color debe variar según el ángulo o la distancia de la línea respecto al centro.

Requisitos técnicos

1. Debe reutilizar la función `bresenham()` tal como fue presentada en clase, sin modificar su firma.
2. Debe implementar una función `generar_puntos_circulo(cx, cy, radio, n)` que devuelva una lista de *n* puntos equiespaciados sobre la circunferencia (use `math.cos` y `math.sin`).
3. Debe implementar una función `dibujar_roseta(pixels, puntos, ancho, alto)` que trace todas las líneas entre pares de puntos, aplicando el gradiente de color.
4. El archivo final debe llamarse `ejercicio2_roseta.py` y debe generar como salida la imagen `roseta.png`.
5. El programa debe generar **tres variantes** de la roseta con distinto valor de N (por ejemplo: N=12, N=24, N=36) y guardarlas como `roseta_12.png`, `roseta_24.png` y `roseta_36.png`.

Pista de arranque

```
# ejercicio2_roseta.py
from PIL import Image
import math

def bresenham(pixels, x0, y0, x1, y1, color, ancho, alto):
    # ... (copiar la implementación vista en clase)
```

```
def generar_puntos_circulo(cx, cy, radio, n):
    puntos = []
    for i in range(n):
        angulo = 2 * math.pi * i / n
        x = cx + int(radio * math.cos(angulo))
        y = cy + int(radio * math.sin(angulo))
        puntos.append((x, y))
    return puntos

def dibujar_roseta(pixels, puntos, ancho, alto):
    n = len(puntos)
    for i in range(n):
        for j in range(i + 1, n):  # conectar cada par
            # calcular color segun el angulo o indice
            color = (...)
            bresenham(pixels, puntos[i][0], puntos[i][1],
                      puntos[j][0], puntos[j][1], color, ancho, alto)

# Generar las tres variantes
for n in [12, 24, 36]:
    img = Image.new("RGB", (700, 700), "white")
    pixels = img.load()
    puntos = generar_puntos_circulo(350, 350, 300, n)
    dibujar_roseta(pixels, puntos, 700, 700)
    img.save(f"roseta_{n}.png")
```

Resultado esperado

Tres imágenes PNG de 700 × 700 píxeles con rosetas de distinta densidad. La roseta con N=12 será más simple y angular; la de N=24 mostrará patrones más intrincados; la de N=36 producirá un efecto casi de encaje. Las tres deben mostrar el gradiente de color aplicado.

> ⭐ **Extensiones voluntarias (hasta 2 puntos adicionales)**
>
> Genere una cuarta roseta donde las líneas se conecten SOLO con puntos no adyacentes (por ejemplo, saltando de a 5 puntos).
>
> Experimente con fondos de color y colores de línea complementarios.
>
> Agregue un script de animación que genere 20 rosetas con N creciente (de 5 a 24) y las combine en un GIF.

Proceso de entrega

La entrega de esta tarea tiene dos componentes complementarios: la **publicación del código** en un repositorio de control de versiones (GitHub o GitLab) y la **entrega formal en Canvas** mediante un documento Word que incluya el enlace al repositorio y las imágenes generadas comprimidas.

Paso 1 · Preparar el repositorio Git

1. Cree una cuenta en `github.com` o `gitlab.com` si aún no tiene una. Ambas plataformas son gratuitas para repositorios públicos y privados.
2. Cree un repositorio nuevo con el nombre `taller5g-unidad3-apellido` (reemplace «apellido» por su apellido en minúsculas, sin tildes ni espacios). Por ejemplo: `taller5g-unidad3-gimenez`.
3. El repositorio puede ser **público o privado**. Si es privado, deberá dar acceso al docente agregándolo como colaborador con usuario `alberto-gimenez` (indíquelo en el documento de entrega si usa otro usuario distinto).
4. Suba al repositorio los siguientes archivos:

- `ejercicio1_casa.py` — código fuente del Ejercicio 1.
- `ejercicio2_roseta.py` — código fuente del Ejercicio 2.
- `casa.png` — imagen generada por el Ejercicio 1.
- `roseta_12.png`, `roseta_24.png`, `roseta_36.png` — imágenes generadas por el Ejercicio 2.
- `README.md` — descripción breve del repositorio (nombre, materia, apellido del alumno, cómo ejecutar los scripts).

> **Comandos Git básicos para subir el código**
>
> cd ~/Desktop/TallerCertificacion1
>
> git init
>
> git add .
>
> git commit -m "Entrega tarea Unidad III"
>
> git branch -M main
>
> git remote add origin https://github.com/USUARIO/taller5g-unidad3-APELLIDO.git
>
> git push -u origin main

Paso 2 · Preparar el documento Word de entrega

El documento Word (.docx) debe contener los siguientes elementos:

1. Encabezado con nombre completo, matrícula, materia, docente y fecha.

2. Enlace al repositorio Git (**URL completa** al cual el docente pueda acceder desde el navegador).
3. Breve descripción de la solución implementada (máximo 1 párrafo por ejercicio, explicando qué estrategia siguió).
4. Capturas de pantalla o inserción de las 4 imágenes generadas (casa.png, roseta_12.png, roseta_24.png, roseta_36.png).
5. Observaciones personales: dificultades encontradas, aprendizajes, extensiones voluntarias implementadas (si corresponde).

Paso 3 · Comprimir y subir a Canvas

1. Cree una carpeta con el nombre apellido_tarea_u3 (por ejemplo: gimenez_tarea_u3).
2. Coloque dentro: el documento Word, las 4 imágenes PNG y los 2 scripts Python.
3. Comprima la carpeta en formato **ZIP**. En macOS: clic derecho sobre la carpeta → «Comprimir». Se generará `apellido_tarea_u3.zip`.
4. Suba el archivo `.zip` al espacio asignado para esta tarea en la plataforma Canvas, en la asignatura Taller de Programación de 5.ª Generación I.

> ⚠ **Verificación antes de enviar**
>
> Confirme que el enlace al repositorio Git funciona desde una ventana de navegación privada (incognito) o pida a un compañero que lo abra. Si el enlace está roto o requiere permisos no otorgados, el trabajo no podrá ser corregido.
>
> Confirme que los scripts Python en el repositorio se ejecutan sin errores en un entorno limpio. Mejor aún: pídale a un compañero que descargue su repositorio y lo ejecute antes de la entrega.

Criterios de evaluación

Cada ejercicio será evaluado sobre 10 puntos según los siguientes criterios:

Ejercicio 1 — Composición con DDA (10 pts)

Criterio	Descripción	Puntos
Funcionalidad	El script se ejecuta sin errores y genera la imagen casa.png	2 pts
Elementos solicitados	La casa incluye los 6 elementos requeridos	3 pts
Uso correcto de DDA	Reutiliza la función dda() sin modificaciones a la firma	2 pts
Modularización	Cada elemento está implementado como función separada con nombre descriptivo	2 pts
Calidad del código	Comentarios claros, nombres de variables adecuados, uso de colores variados	1 pt

Ejercicio 2 — Roseta con Bresenham (10 pts)

Criterio	Descripción	Puntos
Funcionalidad	El script genera las 3 imágenes de roseta sin errores	2 pts
Uso correcto de Bresenham	Reutiliza la función bresenham() sin modificaciones a la firma	2 pts
Distribución de puntos	Función generar_puntos_circulo() implementada correctamente	2 pts
Gradiente de color	Las líneas aplican un gradiente visible y coherente	2 pts
Variantes generadas	Las 3 rosetas (N=12, N=24, N=36) se generan correctamente	1 pt
Calidad del código	Comentarios, modularización, legibilidad	1 pt

Criterios transversales de descuento

Los siguientes aspectos pueden descontar puntaje aunque el código funcione:

- Entrega sin repositorio Git o con enlace roto: **-5 puntos** sobre el total.
- Entrega sin documento Word o sin imágenes: **-3 puntos**.
- Copia detectada entre entregas de distintos alumnos: **entrega anulada** para todos los involucrados.
- Entrega fuera de plazo: **-2 puntos por cada día hábil de retraso**.

Sobre el uso de IA generativa

Está permitido usar herramientas de IA (ChatGPT, Claude, Copilot, Gemini, etc.) como asistente para resolver dudas conceptuales, pero el código final debe ser escrito y comprendido por el estudiante. En las pruebas prácticas del parcial se evaluarán los mismos conceptos; quien delegue 100 % el trabajo a la IA tendrá dificultades para aprobar el examen.

Buena práctica: pedir a la IA que explique un error, no que escriba el código completo. El objetivo de la tarea es ejercitar la programación propia.

Recursos y soporte

Códigos fuente de referencia

Los códigos fuente de las funciones `dda()` y `bresenham()` están disponibles en:

- El material de lectura de la Unidad III (secciones 2.2 y 2.3).
- Las presentaciones de las Clases 6 (diapositivas 11 y 15).
- Los scripts de demostración ejecutados en clase (los alumnos recibieron copia por Canvas).

Tutoriales recomendados

- Pillow (PIL) — Documentación oficial: `https://pillow.readthedocs.io/`.
- Git y GitHub — Tutorial gratuito: `https://docs.github.com/es/get-started`.
- GitLab — Guía para principiantes: `https://docs.gitlab.com/ee/tutorials/`.

Prof. Mgtr. Alberto F. Giménez Méndez

Docente titular

Taller de Programación de 5.ª Generación I

Universidad Americana · Asunción, Paraguay · 2026